	ORIGINATE DATE	EDIT DATE	DOCUMENT-REV. NUM.	PAGE
	13 November, 2000	4 September, 2015	GEN-CXXXXX-REVA	1 of 16

Author: Steve Morein

Issue To:	**Copy No:**

R400 Architecture Proposal

ver 0.1

Overview: The is a proposal for the overall architecture of the R400. It is also just a proposal, and nothing is decided yet.

AUTOMATICALLY UPDATED FIELDS:
Document Location: VST FireWire HD:r400 spec
Current Intranet Search Title : R400 Top Level Spec

APPROVALS	
Name/Dept	Signature/Date

Remarks:

Table Of Contents

Revision Changes:

Rev 0.0 (Steve Morein) Date: November 6, 2000 Initial revision.	Document started
Rev 0.01 Steve Morein Date: November 10,2000	Document continued

Introduction

This document outlines a proposal for the r400 architecture.

A minor note: in the middle of writing this I decided that it makes the most sense to call the “pixel” pipelines shader pipeline since they handle vertices and pixels. I have not gone through this to make sure that my usage is consistent.

1. Features

1.1 AGP8x and possibly serial AGP

The R400 will at a minimum support AGP4x and AGP8x interfaces. We may also support 3.3V i/o including AGP2x and 3.3V PCI. We need to consider how we interface to LDT (AMD) and possibly the Motorola rapid I/O that may be used in future Apple Designs (G5).

1.2 128 Bit memory interface

We are thinking of only supporting a 128 bit interface to memory. The memory will be configured as four channels of 32 bits each. The atomic fetch until will be 256 bits in expectation that some high speed memories will use a prefetch-8 architecture. Logic in the memory controller will optimize down to 128 bit writes when possible on DDR or prefetch 4 memories. Memories up to 500 MHz will be supported (1 gigabit data rate).

Memory is the most open issue on the R400. We need to develop a roadmap ASAP for how memory will develop, and this may significantly affect our plans.

1.3 Nearly transparent dual chip

To be able to address the very high end desktop/enthusiast market we will support a glueless two chip design instead of a 256 bit bus. Unlike previous dual chip designs we have done, this is targeted to be a mainstream product. This implies that it can easily be WHQL'd, and can accelerate all applications and benchmarks, not just a subset of full screen apps. A separate document outlines the two proposals we are looking at for the dual chip design.

There will be costs added to the base chip to support this. Design time, pins, and area will be impacted by adding this support.

1.4 Unified processing pipe

The most ambitious feature in this design is the "truly unified pipe" : a single programmable pipeline is used for 2D, Video, 3D vertex, and 3D pixel operations. The unified pipeline does all of its calculations in 32 bit floating point, the same as the existing vertex transform in previous chip, and the next step in the precision of the color/pixel calculations which have increased from 8 bits (R100), through 16 bits (R200), to the 20 bits in the R300.

There is an area cost to the unified pipeline since we are forced to go to 32 bit precision for color, when application requirements may need less (22 to 24 bits). However the unified pipeline results in a single math/register structure compared to the separate structures in a more traditional design. It is hoped that by only needing to design the one structure we can make the investment in design time and effort to really optimize the area.

Some of the benefits to merging the pipelines include allowing the vertex operations to do texture fetches, which we could not afford add logic to the transform pipe to do, a single programming model for both operations, more precision on color than we would normally provide, and the ability to support significantly more registers and instructions in pixel shaders.

One important benefit is load balancing. In the current pipeline when the app it transform bound the pixel pipeline is idle some significant portion of the time, and when the app is raster bound the transform hardware idle. The unified pipeline presented here dynamically allocates its processing power between transform and raster.

1.5 Front end scaling

We will remove the back end scaling capability from the display, and replace it with a non-scaling overlay. This will require us to be able to implement scaling using the unified pipeline. Key features that will need to be supported are large filter kernels, de-interlacing, frame rate conversion, and good support for YUV and color conversion.

1.6 Control processor

To allow us to emulate a backend scaler and to enable new applications the control processor will be enhanced with event based streams. These are secondary, real time, command streams that start execution when an event happens.

1.7 Real-Time drawing command ability

To allow for the emulation of backend scaling as well as support new features we need to be able to interrupt the 3D pipe and be able to execute high priority commands with low latency. At the moment it appears far to difficult to be able to insert a new command at the top of the 3D pipeline and meet latency requirements (which I believe we wish to define as around a 1/16 of a frame refresh). This would require us to be able to interrupt triangles in the midst of rasterizing, inset vertices in the midst of a large vertex array, and other nasty things. I think instead we can get by with a second rasterizer which drives the pixel pipelines. Setup would be done with software, but since the majority of the real time rasterization is expected to be simple

1.8 3D Features

There are a number of new 3D features we are considering for inclusion. Additional features may be added, and some of these may be dropped.

1.8.1 *Noise Textures*

Perlin style noise is useful for a number of applications. It is generated on chip and consumes no external memory bandwidth. It also larger than any physical texture can be: 256x256x256 lattice points, and still has detail when the resolution is 4Kx4Kx4K. There is an opportunity to get this adopted as part of dx9.

1.8.2 *Shadow buffers*

John Carmack is using shadow volumes to generate shadow effects in doom3. Shadow volumes are very poor way to use modern 3D pipelines. (will add more detail here later). Shadow buffers have two key limitations: very high resolutions are required to avoid aliasing, and traditional shadow buffers can not be mip-mapped so filtering is real problem. Through a combination of the z techniques we have developed and, hopefully, deep shadow buffers, we can solve both of these problems and widely enable shadow buffers.

1.8.3 *Anti-Aliasing*

We want to further improve the anti-aliasing used in the R300 by reducing the needed memory, and possibly increasing the number of samples per pixel. The goal is more than fifty percent of the performance and less than three times the memory of anti-aliased rendering. We should also look into improved methods.

1.8.4 *Texture compression*

To further reduce bandwidth we need to improve texture compression. We need to achieve both better compression that S3TC, and have a high enough quality that textures that would lose too much detail with S3TC can be compressed. Both of these goals do not need to be achieved simultaneously on all textures. We also need to look at compression of non-traditional surfaces such as normal maps.

1.8.5 *Z compression*

We will build on the R300 slope based compression but we are looking at supporting maxmin for cachelines that do not compress with slopes (either too many slopes per cacheline, or the pixel shader modifies the z value)

1.8.6 *Filtering*

As part of the move to front end scaling we need better than bi-linear filters. Goals are : arbitrary sized separable filters and 4x4 bi-cubic. Being able to support programmable weights is nice.

1.8.7 *Curved Surface Support*

We need to figure out how we are going to support curved surfaces in this architecture. I think that we can find a way to use the wide ability of a vertex shader to implement acceleration for subdivision surfaces, but the vertex only level of processing in the shader pipeline means that something ahead of it needs to set up the surface. At one point I imagined that we could use the sibyte processor as a CP, which would have the power to do the curved surface setup. That is obviously no longer possible.

1.9 High color depth

We will support a 64 bit color buffer (16:16:16:16), the exact format (fixed, floating, etc.?) has yet to be decided

1.10 Performance

I think we can increase the clock speed from 300 MHz to 500MHz.
Historically the goal has been to double speed in each generation, assuming a constant clock speed. However since we are considering the dual chip solution for the very high end we may not need to be 2x the speed of the R300. Our use of a 128 bit memory bus instead of 256 bits will impose a potentially lower bandwidth.

That said I would still like to aim for 2x the internal processing capability of the R300:

	R300	R400
Clock speed	300 MHz	500 MHz
Pixels per clock	8	16
Bi tex fetches per pixel	2	2
ALU ops/clk (mac's)	64 (dedicated)	192 (shared)
Peak Tri/Sec	150	250
Peak xform fp ops/clk	16? (dedicated)	192 (shared)

We may need to reduce this performance goal to meet our area goal.

2. Area

The area goal for the R400 is 10mm on a side in .13 micron CMOS
There will probably be a lower cost version with a target area of 8.5 on a side.

3. Schedule

Tapeout April 2, 2002
Samples May, 2002
Production Nov, 2002

4. Process

At the moment this looks like an easy choice: .13 will be in production for over a year, and .10 does not show up until the very end of 2002 according to the TSMC and UMC roadmaps.

We will probably want to be in a flip chip packaging approach to meet power distribution goals. It will also reduce the cost of the dual chip option by making the extra pins needed for the interface cheaper. Is there a way to have an

option to wire bond it also? Possibly without the dual chip interface, and with less pwr/gnd forcing a lower clock speed. This may make sense for a lower cost sku.

5. Dual Chip

<need to copy the dual chip notes over and add to them>

6. General rendering operation

6.1 Unified Shader

The unified shader is a simd/vector engine that performs the same instructions on four sets of four (16 total) elements. For pixel shader operations the elements are pixels with the sets of four required to be 2x2 footprints. For vertex shader operations the sixteen elements are sixteen vertices. The basic element is a 4 value vector – frequently interpreted as x,y,z,w or r,g,b,a.

The user model for the unified shader is composed of a variable number of general purpose registers, a subset of which are usually initialized with data. An ALU can do simple math, conditional moves, and permutations on the registers, and the ability to do a limited number of memory reads using the texture cache. The number of register is variable, and the number of registers required for an operation are specified when the task is submitted to the unified shader. The unified shader will not start the task until there is enough free room for the tasks registers.

The unified shader is based on the R300 partially unified shader.

6.2 3D Rendering

For 3D rendering data is passed twice through the unified shader- once to transform the vertices and a second time to determine the color of the pixels.

The input to the 3D pipe is expected to be indexed vertex arrays. Linear vertex arrays can easily be supported by the CP generating sequential indices. Inline vertex data is an open issue, I would prefer to write it to memory and then fetch it as a vertex array rather than add a direct path.

The stream of indices is sent to the Primitive Assembly block by the CP. The front of the primitive assembly block maintains the tag for the vertex cache; The vertex cache stores transformed vertices. As misses are detected in the tag, the indices that miss are placed into 16 entry vectors. Each vector contains a state pointer, a pointer to the vertex shader to be used, and the 16 indices to vertices that need to be transformed. When either a vector is filled with 16 entries or a state change happens (so that the next vertex does not share the state and vertex shader with the previous vertex) the vector is issued to one of the “shader” pipelines for transformation. Which of the four shader pipelines it is issued to determined either by some effort of load balancing or a simple round robin. All that is submitted to the pixel pipeline is the state, the vertex program, and the indices. The shader pipeline will fetch the vertex array data through the cache infrastructure that is also used for texture fetches. After the tag the indices (actually now the indices into the vertex cache) are placed into a latency FIFO to hide the latency of transforming the vertices.

The shader pipeline receives the vector of 16 indices from the primitive assembly block. The shader pipeline operates, when rendering pixels, by processing a vector of four 2x2 pixel footprints, A total of 16 pixels. For vertex processing each of the pixels is replaced with a vertex. The vertex program includes information of how many local variables it will need. The rasterizer waits until that many local variables are free, (as each executing thread in the shader pipeline terminates it frees its local variables). With the proposed shader datapath the maximum number of local variables per vertex is 256. However this leaves no ability to hide latency, 16 to 32 local variables will probably maximize latency hiding and therefore performance. The vertex shader program can use all the capabilities of the shader pipeline including texture fetches and dependent lookups. At the end of the vertex program, the transformed coordinates must be output. One output will be the x,y,z,w position which we be stored in the position cache of the vertex cache. The vertex program may also output a number of parameter values (colors, texture coordinates, other

interpolated inputs into the pixel shader). The parameter values must be output as a multiple of four 128 bit words, as the parameter cache is designed for this.

The primitive assembly block reads the indices back out of the latency FIFO and accesses the position cache portion of the vertex cache. It assembles the vertices into primitives (lines, triangles, rectangles, quads?, points, ?). Baricentric values are assigned to the vertices, and will be used later in the rasterizer to interpolate the parameters. The parameters are not accessed by the primitive assembly logic, which only works from the position data. The primitive is clipped against both the viewing volume as well as user clip planes, with fractional baricentric coordinates assigned to the clipped primitive sections. The primitive goes through the perspective divide and the viewport transform. The resulting screen space primitive is setup (plane equations for 1/W, Z, and the baricentric coordinates). The resulting primitive data, including the indices back into the parameter portion of the vertex cache are broadcast to the four pipes. The final time that an index is output that access the oldest vertex cache line, a token is also sent. When all of the four pipelines return the token the primitive assembly block can free that cacheline and allow it to be used for a new vector of vertices. The performance goal in the primitive assembly block is a triangle every two clocks.

Each pipe has a FIFO in front of the rasterize to load balance. Each pipe will handles 16x16 sections which are interleaved between the pipes. To maximize the effective size of the FIFO we will probably cull the triangle list before the FIFO. The rasterizer will request the parameter data from the parameter cache for the primitives. A small latency hiding FIFO will hide the latency of the access to the parameter cache. The parameter cache is 512 bits wide, and the interfaces from the parameter cache to the rasterizer are 128 bits wide, this allows the parameter cache to output one pipelines request per clock, which is serialized over four clocks, keeping all four interfaces busy. The rasterizer keeps a small cache of three to four vertices, this allow only the new parameter to be fetched when adjacent triangles are processed. The parameter cache interface imposes a second performance limits, in the worst case each polygon covers all four pipelines and there are no vertices shared from triangle to triangle. In this case the peak performance is (500 MHz / (4 pipelines * 3 vertices) = (500/12) = 41.6 million triangles per second. In the best case triangles are perfectly stripped and never cross over pipeline boundaries. In this case the peak performance (If we ignore the setup limit) is 500 million triangles per second. As a practical manner we should be able to approach the setup limit of 250 Million triangles per second.

The rasterizer also contains a portion of the hierarchical Z memory. We are looking into moving this into a cache based approach, but that is far for certain at this point. We would like to be able to do heirz culling at a speed in excess of 64 pixel per clock per pipelines (256 pixels per clock total). We are also going to consider some of the improved latency heriz options to improve culling efficiency.

The rasterizer will generate four pixels per clock if there are no more than eight interpolated parameters. The rasterizer generates vectors of four 2x2 footprints (16 pixels). Each 2x2 footprint must be screen aligned and from the same triangle (with a single shared z slope). The four footprints only need to share the same state and shader program.

Before starting the processing of a vector the rasterizer (which includes the sequencer for the shader pipeline) checks to make sure that there are enough free registers in the shader pipeline for the pixel shader program. If not, it stalls until there are enough. The rasterizer also needs to arbitrate between the three streams of vectors to be shaded: the vertex stream, the pixel stream, and the real time stream. I think it will be sufficient for the real time stream to have priority over the vertex stream which has priority over the pixel stream. This will meet the realtime demands, and keep the vertex cache filled.

The vector is then processed by the shader pipeline. We will probably support up to eight sequentially dependent texture fetches. (to use the R300 terminology, eight clauses). 16 (8?) textures are supported, but each texture can be accessed multiple times by a single pixel shader which can provide a different address each time. This is especially useful for complex filters.

The output of the pixel shader is the final color of the fragment. The pixel shader may also replace the Z value. Fog and stippling must be done in the pixel shader program.

The render backend does the z compare, stencil operation and color alpha blend.

The texture fetch path has a number of design options. One option is an approach where the local, multiported, texture cache is small (1 to 4 KB), and contains uncompressed color in a canonical format (32 bits per pixel) and uses a 4x2 or 4x4 cacheline. This is backed up by a large (>16KB) L2 cache which also stored uncompressed 8x8 cachelines. The decompression logic lives between the memory controller and the L2 cache.

An alternative design uses the L2 cache to contain data in memory format (compressed) which is decompressed as needed to fulfill L1 texture cache misses. This will increase the effective size of the L2. The L2 cache is distributed, with 1/4 of it residing in each memory controller. The Texture decompression logic can either be located in each shader pipeline, or exist as a shared block(s) that receive data from all four memory controller and send the decompressed 4x4 cachelines to each shader pipeline. The unified decompression block will result in better performance, and possibly less area, at the cost of some of the scalability.

Assuming that we chose the L2 in memory controller and the unified decompression logic, the texture path would work as follows:

In a four pipeline design there are two texture decompression blocks, one for the “left” texture units in each shader pipeline, and the second for the “right” texture units. In the two pipeline, lower cost, version of the chip only a single decompression pipeline is used, serving the left and right texture units.

The L1 texture cache receives a texture request from its shader pipeline. The usual tag and latency FIFO is used to generate the misses. These are sent to the shared texture decompression block, which looks up the texture to find the physical address and then sends the request to the L2 cache in the memory controller. The L2 also has a latency FIFO and tag, and will return the data in order (but there is no order guaranteed between the data returning from each L2). The decompression block has a buffer which is used to place the data from the memory controllers back in order. The decompression logic decompresses the texture and returns, in order, the 4x4 cachelines that the L1 caches are requesting. Most of the compression techniques we are considering are based on an 8x8 tile (or 4x4x4), when necessary the decompression logic will decompress an entire 64 pixel tile and only return the requested 16 pixels to the L1 cache. This will tend to increase the bandwidth between the decompression logic and the L2 cache as 8x8 blocks are repeatedly requested to provide different 4x4 subtiles to the L1. The L2 cache will prevent the repeated reads from going to memory, and we will probably implement an “L0” style cache in front of the L2 to also catch the redundant requests.

Each memory controller will have two 64 bit read return buses, one to each of the two decompression blocks, each decompression blocks drives a separate 128 bit bus to each of the four shader pipelines. This will tend to have better utilization and load balancing than having the memory controller drive a 32 bit bus to the decompression logic in each shader pipeline. While the total number of wires is similar (128 bits per memory controller, 128 bits into each texture cache) we are less likely to leave the texture pipes starved when there is some imbalance.

6.3 2D Rendering

2D rendering is implemented in the 3D pipeline. The first reason for the change is performance; The current 2D pipeline can render 128 bits per clock (16 8 bit pixels, 8 16 bit pixels, 4 32 bit pixels). The 3D pipe can render 16 pixels per clock, and the pixels can be 8 to 64 bits wide. Secondly routing the three busses needed by the 2D engine (src read, dest read, dest write) has a certain cost, and complicated the design of the chip. If we attempt to improve the performance of the 2D pipe these busses will increase in size, further complicating the design. Another reason is dual chip rendering, it would be nice if 2D as well as 3D operations increase in speed. (2D operations include things like color clears and texture uploads that do show up in 3D benchmarks.

There are four issues currently known that will affect placing the 2D commands into the 3D pipelines:
1) Command compatibility
2) Hostpath blits
3) ROP3
4) Overlapping blits.

We wish to be as compatible as possible with the existing PM4 2D model. This will require that the CP be enhanced, allowing it to translate 2D commands into commands understood by the 3D pipe. We will ad interfaces to the 3D pipe to make this easier but there will still be significant amount of work that still needs to be done by the CP.

Host path blits can no longer work as they do now. Four pipelines will be attempting to execute the requested command in parallel, walking the area to be drawn in some tightly tiled pattern to optimize memory and cache performance. This bears little resemblance to the single linear stream of data from embedded into the PM4 command stream. In addition the shader pipelines are heavily optimized for a pull, “reverse” mapping model, and not a push, “forward” model. The basic solution to this problem is for the 3D pipes to pull the source data directly out of a

command/data buffer, in whatever order, and in whatever parallel streams exist. Whether we give the PM4 engine the ability to skip over the hostpath data, or force the driver to move hostpath data into a second command ring still needs to be decided. It is possible that one or more changes may be needed to the driver for this.

2D supports a ROP3 operation that requires the destination color as well as two sources: the source, and the pattern. To support this the pattern color is output by the pixel shader on the Z output which is not used by 2D operations. The render backend now has all three needed sources.

Overlapping blits is an ugly problem that I have not yet found an acceptable solution to. More work needed.

One benefit to these changes is the 2D operations will also be accelerated by the second chip in a dual chip board.

6.4 Real-time Rendering

7. Display operation

The display must be able to display from microtiled surfaces and overlays. This will generally force us to adopt line buffers.

The display should support at least two outputs, ideally we will be able to support two high resolution outputs and a low resolution output (TV out)

We will drop support for overlay scaling, and therefore supporting an overlay on all displays becomes affordable, fixing a “bug” that our current dual display products suffer from.

We will place the overlay line buffers in the memory controllers, this changes the interface from the memory controllers to the display from a wide “bursty” interface to a narrow continuos interface.

<this rest of this was copied out of another document and needs some editing to fit in this document. Bear with us as construction of this document continues>

Video buffer operation:

At beginning of even scanline (scanline 0) 1/2 of line buffer is filled.
The upper half of the buffer is read out, at the same time as the second half of the buffer is filled. When the scanout reaches the midpoint 3/4 of the buffer is filled. When the scanout reaches the end of the scan the buffer is filled. The speed at which the buffer is filled must be greater than 1/2 of the rate at which the buffer is scanned out.

For the odd scanlines, the buffer is completely filled at the start of scanout (as a result of the even scanline finishing properly). As the lower half is scanned out, reads are issued to fetch the data for the next pairs of scanlines. At the end of the odd scanline, the buffer is expected to contain half of the data for the next scanline.

Another way of looking at this is as follows:

At the beginning of the odd scanline the scan buffer is filled. As each word is read from the buffer and sent to the display logic, a request is made to the memory controller to fill in the data. It is not necessary for all the data for the next scanline pair to be fetched by the time that the scanline reaches the end, the real requirement is for the last word in the scanline buffer to get there just before it is read, at the end of the next even scanline.

We will support a single non-scaled overlay per each display.

Some bandwidth numbers
In reality we do not need to deal with quite as much bandwidth as the FIFO in the display can hide the horizontal retrace.
350 MHz primary fetch, 32 bit data:
350 MHz primary display, 32 bits
350 MHz overlay fetch

350 MHz overlay display
total: 350 MHz * 16 bytes (128 bits)

Since we support dual monitor, this is doubled.

One design option is to split the display scanline into pieces and move them into the memory controller. This greatly reduces the exposed bandwidth in the system (reducing power and routing problems)

If we assume that there are four memory controllers, each with 2GB/s of memory bandwidth then the following will work:

core clock speed: >= 1x the memory clock
memclk = 500 MHz
coreclk = 500 MHz

each memory interface is 32 bits at a DDR rate, and the fetch granularity is 256 bits.
Therefore if data was continuously received into the display FIFO 64 bits would be received every clock. A 256 bit interface at the core clock rate is more than adequate..

the memory size needed for two 2048 displays is : 2048*4 bytes * 2 scanlines * 4 buffers is 64KB. So each buffer is 16KB (128 Kbit). With a 256 bit interface the memory is 256x512 single ported.

For writes into the write buffer as a result of memory fetches, a small buffer reorders data between pairs of 256 bit words so that while what is read from memory is 256 bits containing two vertically stacked 128 bit words, what is written is two 128 bit words that are on the same scanline.

The interface from the memory controller to the display only needs to be big enough for the sustained bandwidth, and not the peak memory speed bandwidth. A 16 bit interface to each display seems like more than enough.
(compare this to the rage 6 with two 128 bit busses between the memory controller and display)

A couple more notes:
for the most part the memory format is stored in the scanline buffer. The exception is 64 bit, which we would like to convert to something like 11:11:10 or 8:8:8:8 This may mean some sort of gamma circuit in the memory controller.

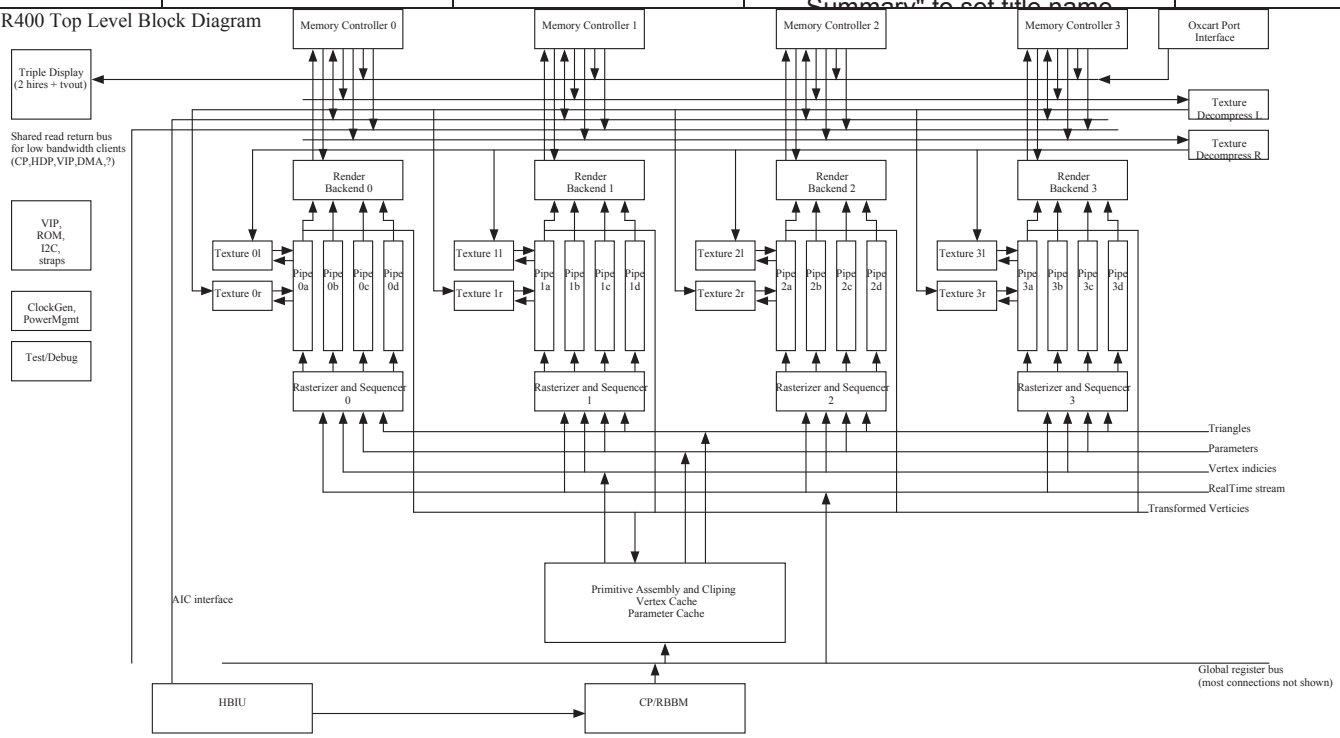
A LUT would exist in the display for gamma de-correction and pallet support.

The interleave between the memory controllers would have to be compatible with the tiling and still give good performance.

A big question is how does this work in a two chip board? I had been thinking about interleaving on a fine basis between the chips with a display controller in one chip fetching from both, but this somewhat flips that around. We need to route the video signals as an extra channel between the chips, this will add complexity, but it actually is less bandwidth since the overlay is combined first.

8. Block diagram

R400 Top Level Block Diagram



9. Short Block descriptions

9.1 SYS

The system blocks support the chip, but are not graphics specific.

9.1.1 *HBIU*

The HBIU is the interface to the host bus. It implements four interfaces: register/read write, HDP for host access to memory and the

9.1.2 *HDP*

9.1.3 *MISC*

9.1.4 *Rom*

9.1.5 *VIP*

9.1.6 *I2C*

9.1.7 *?*

9.1.8 *ClockGen*

9.1.9 *CP*

9.1.10 *RBBM*

9.1.11 *MC*

The memory controller is distributed, each of the four memory channels has a separate memory conttoller. Each memory control contain a part of the L2/Line buffer memory. This large buffer serves a number of purposes in the graphics chip, including L2 cache for textures and verticies.

9.2 Display

9.3 Grfx

9.3.1 *PrimitiveAssembly/vertex cache*

9.3.2 *Raster Engine*

9.3.3 *Sequencer*

9.3.4 *Datapath*

9.3.5 *TextureEngine*

9.3.6 *RenderBackend*

10. Top Level Interconnections

10.1 First Level Sub Heading

10.1.1 *Second Level Sub Heading'*